Лекция 8: ленивость

Функциональное программирование на Haskell

Алексей Романов

9 октября 2026 г.

МИЭТ

Передача по значению

- Как вычисляется выражение вида `f(g(x, y), h(z))` в привычных языках?

Передача по значению

- Как вычисляется выражение вида `f(g(x, y), h(z))` в привычных языках?
- Сначала вычисляются аргументы `g(x, y)` и `h(z)` (гарантированно в этом порядке или нет), потом их значения передаются в `f`.
- Это называется *передачей аргументов по значению (call-by-value)*.
- Для вычисления выражения нужно вычислить все подвыражения.
- Исключения в C-подобных языках?

Передача по значению

- Как вычисляется выражение вида `f(g(x, y), h(z))` в привычных языках?
- Сначала вычисляются аргументы `g(x, y)` и `h(z)` (гарантированно в этом порядке или нет), потом их значения передаются в `f`.
- Это называется *передачей аргументов по значению (call-by-value)*.
- Для вычисления выражения нужно вычислить все подвыражения.
- Исключения в C-подобных языках?
- `&&`, `||`, `? :`.

Передача по значению

- Как вычисляется выражение вида `f(g(x, y), h(z))` в привычных языках?
- Сначала вычисляются аргументы `g(x, y)` и `h(z)` (гарантированно в этом порядке или нет), потом их значения передаются в `f`.
- Это называется *передачей аргументов по значению (call-by-value)*.
- Для вычисления выражения нужно вычислить все подвыражения.
- Исключения в C-подобных языках?
- `&&`, `||`, `? :`.
- В них вычисляется первый операнд, а второй (и третий для `? :`) — только если необходимо.

Передача по имени: Scala

- Как можно сделать по-другому?

Передача по имени: Scala

- Как можно сделать по-другому?
- В Scala параметр `x: => Int` передаётся *по имени (call-by-name)*: выражение вычисляется при каждом использовании.

```
def twice(x: => Int): Int = x + x

twice({ println("evaluating"); 2 })
```

Передача по имени: Scala

- Как можно сделать по-другому?
- В Scala параметр `x: => Int` передаётся *по имени (call-by-name)*: выражение вычисляется при каждом использовании.

```
def twice(x: => Int): Int = x + x

twice({ println("evaluating"); 2 })
// дважды печатает evaluating, возвращает 4
```

Передача по имени: Scala

- Как можно сделать по-другому?
- В Scala параметр `x: => Int` передаётся *по имени (call-by-name)*: выражение вычисляется при каждом использовании.

```
def twice(x: => Int): Int = x + x

twice({ println("evaluating"); 2 })
// дважды печатает evaluating, возвращает 4
```

- Если параметр не используется, выражение не вычисляется. Результат автоматически не запоминается.
- Имена в аргументе сохраняют привязки из места вызова.
- By-name Parameters: документация Scala.

Передача путём макроподстановки

- Макросы в C — каждое использование аргумента заменяется на переданное выражение (а не на его значение).
- И это повторяется: макросы в результате подстановки тоже будут заменены на свои определения.
- Это *передача путём макроподстановки (call-by-macro-expansion)*.

Передача путём макроподстановки

- Макросы в C — каждое использование аргумента заменяется на переданное выражение (а не на его значение).
- И это повторяется: макросы в результате подстановки тоже будут заменены на свои определения.
- Это *передача путём макроподстановки (call-by-macro-expansion)*.
- Похожа на передачу по имени, но подстановка может менять смысл выражения из-за приоритетов операций и захвата имён.

Сравнение на примерах 1

- Если рассмотреть

  ```
  (\x -> x + x) (2 + 2)
  ```

  то при передаче по значению будет вычислено

Сравнение на примерах 1

- Если рассмотреть

  ```
  (\x -> x + x) (2 + 2)
  ```

  то при передаче по значению будет вычислено

  ```
  (\x -> x + x) 4  ⇝  4 + 4
  ```

  а при передаче по имени —

Сравнение на примерах 1

- Если рассмотреть

  ```
  (\x -> x + x) (2 + 2)
  ```

  то при передаче по значению будет вычислено

  ```
  (\x -> x + x) 4  ⇝  4 + 4
  ```

  а при передаче по имени —

  ```
  (2 + 2) + (2 + 2)
  ```

Сравнение на примерах 2

- `if1 x y z = if x then y else z`

  `if1 True 1 undefined`

  `undefined :: a` вызывает ошибку при вычислении; `error "сообщение"` позволяет задать её текст.

Сравнение на примерах 2

- `if1 x y z = if x then y else z`

  `if1 True 1 undefined`

  `undefined :: a` вызывает ошибку при вычислении; `error "сообщение"` позволяет задать её текст. Что случится при передаче по значению? А по имени?

Сравнение на примерах 2

- `if1 x y z = if x then y else z`

  `if1 True 1 undefined`

  `undefined :: a` вызывает ошибку при вычислении; `error "сообщение"` позволяет задать её текст. Что случится при передаче по значению? А по имени?
- По значению: ошибка.

Сравнение на примерах 2

- `if1 x y z = if x then y else z`

  `if1 True 1 undefined`

  `undefined :: a` вызывает ошибку при вычислении; `error "сообщение"` позволяет задать её текст. Что случится при передаче по значению? А по имени?
- По значению: ошибка.
- По имени: `1`.

Какой порядок лучше?

- Плюсы передачи по имени:

Какой порядок лучше?

- Плюсы передачи по имени:
  - Можно избежать ошибок.
  - Можно избежать лишних вычислений

Какой порядок лучше?

- Плюсы передачи по имени:
  - Можно избежать ошибок.
  - Можно избежать лишних вычислений (представьте, что в предыдущем примере вместо `undefined` вызов сложной функции).

Какой порядок лучше?

- Плюсы передачи по имени:
  - Можно избежать ошибок.
  - Можно избежать лишних вычислений (представьте, что в предыдущем примере вместо `undefined` вызов сложной функции).
- Главный минус:

Какой порядок лучше?

- Плюсы передачи по имени:
  - Можно избежать ошибок.
  - Можно избежать лишних вычислений (представьте, что в предыдущем примере вместо `undefined` вызов сложной функции).
- Главный минус:
  - Можно сделать очень много лишних вычислений.

Какой порядок лучше?

- Плюсы передачи по имени:
  - Можно избежать ошибок.
  - Можно избежать лишних вычислений (представьте, что в предыдущем примере вместо `undefined` вызов сложной функции).
- Главный минус:
  - Можно сделать очень много лишних вычислений.
  - Как только мы используем какую-то переменную более одного раза

Какой порядок лучше?

- Плюсы передачи по имени:
  - Можно избежать ошибок.
  - Можно избежать лишних вычислений (представьте, что в предыдущем примере вместо `undefined` вызов сложной функции).
- Главный минус:
  - Можно сделать очень много лишних вычислений.
  - Как только мы используем какую-то переменную более одного раза (если переменные в её определении не изменились).

Передача по необходимости

- В Haskell используется *передача по необходимости (call-by-need)*:
- Как передача по имени, но значение каждой переменной вычисляется только один раз.

Передача по необходимости

- В Haskell используется *передача по необходимости (call-by-need)*:
- Как передача по имени, но значение каждой переменной вычисляется только один раз.
- Это имеет смысл потому, что все переменные неизменяемы.

Передача по необходимости

- В Haskell используется *передача по необходимости (call-by-need)*:
- Как передача по имени, но значение каждой переменной вычисляется только один раз.
- Это имеет смысл потому, что все переменные неизменяемы.
- При этом значения тоже не обязательно вычислять «до конца»: мы хотим, чтобы `length [undefined, undefined, undefined]` возвращало `3`.

Передача по необходимости

- В Haskell используется *передача по необходимости (call-by-need)*:
- Как передача по имени, но значение каждой переменной вычисляется только один раз.
- Это имеет смысл потому, что все переменные неизменяемы.
- При этом значения тоже не обязательно вычислять «до конца»: мы хотим, чтобы `length [undefined, undefined, undefined]` возвращало `3`.
- Ленивость в том или ином виде есть во многих языках (пример: `System.Lazy<T>` в .NET).
- Но в Haskell она встроена очень глубоко.

Нормальная форма

- Выражение находится в нормальной форме, если там нет ничего, что можно вычислить (редуцировать).
- Примеры:

Нормальная форма

- Выражение находится в нормальной форме, если там нет ничего, что можно вычислить (редуцировать).
- Примеры:

```
42
(2, "hello")
\x -> (x + 1)
```

- Примеры выражений не в нормальной форме:

Нормальная форма

- Выражение находится в нормальной форме, если там нет ничего, что можно вычислить (редуцировать).
- Примеры:

```
42
(2, "hello")
\x -> (x + 1)
```

- Примеры выражений не в нормальной форме:

```
1 + 2
(\x -> x + 1) 2
"he" ++ "llo"
(1 + 1, 2 + 2)
let x = 1 in x
```

Теоремы о нормальных формах в лямбда-исчислении

- В лямбда-исчислении можно доказать:
- Два порядка вычисления одного выражения не могут привести к разным значениям (нормальным формам).
- Но может случиться, что один из них даёт значение, а другой — нет.
- Если нормальная форма существует, нормальный порядок редукции найдёт её.
- Нормальный порядок: сначала самый левый внешний редекс, в том числе под лямбдами.
- Ленивое вычисление останавливается раньше: когда получена нужная часть результата. Под лямбдами оно не вычисляет.

Слабая заголовочная нормальная форма

- Выражение находится в слабой заголовочной нормальной форме (WHNF), если это:
  - Лямбда.
  - Литерал.
  - Конструктор данных, возможно с аргументами.
  - Частичное применение функции, которому для выполнения её тела ещё нужны аргументы.

  Аргументы здесь — любые выражения

Слабая заголовочная нормальная форма

- Выражение находится в слабой заголовочной нормальной форме (WHNF), если это:
  - Лямбда.
  - Литерал.
  - Конструктор данных, возможно с аргументами.
  - Частичное применение функции, которому для выполнения её тела ещё нужны аргументы.

  Аргументы здесь — любые выражения (а для НФ они тоже должны быть в НФ).
- Примеры:

Слабая заголовочная нормальная форма

- Выражение находится в слабой заголовочной нормальной форме (WHNF), если это:
  - Лямбда.
  - Литерал.
  - Конструктор данных, возможно с аргументами.
  - Частичное применение функции, которому для выполнения её тела ещё нужны аргументы.

  Аргументы здесь — любые выражения (а для НФ они тоже должны быть в НФ).
- Примеры:

```haskell
(1 + 1, 2 + 2)        -- (,) (1 + 1) (2 + 2)
\x -> 2 + 2 + x       -- не НФ: 2 + 2 под лямбдой
undefined : take 4 [1..]
(+) 1
```

Отложенные вычисления

- Отложенное вычисление (thunk) хранит выражение и окружение; в схемах обозначим его $\langle e \rangle$.
- Если мы напишем
  ```
  x = (length [1,3], undefined)
  ```
  то x может указывать на отложенное вычисление или уже на пару: это зависит от оптимизаций.
- После этого
  ```
  case x of
    (y, z) -> ...
  ```
  требует привести x к WHNF. Схематически:

Отложенные вычисления

- Отложенное вычисление (thunk) хранит выражение и окружение; в схемах обозначим его $\langle e \rangle$.
- Если мы напишем

  ```
  x = (length [1,3], undefined)
  ```

  то x может указывать на отложенное вычисление или уже на пару: это зависит от оптимизаций.
- После этого

  ```
  case x of
    (y, z) -> ...
  ```

  требует привести x к WHNF. Схематически:

  ```
  x: (*thunk1*, *thunk2*)  *thunk1*: ⟨length [1,3]⟩
  y: *thunk1*              *thunk2*: ⟨undefined⟩
  z: *thunk2*
  ```

Отложенные вычисления (продолжение)

- Если это

```
case x of
  (y, z) -> print y
```

то y тоже потребуется вычислить, и получим

Отложенные вычисления (продолжение)

- Если это

```
case x of
  (y, z) -> print y
```

то y тоже потребуется вычислить, и получим

```
x: (2, *thunk2*)          *thunk1*: 2
y: 2                      *thunk2*: ⟨undefined⟩
z: *thunk2*
```

:sprint и :print

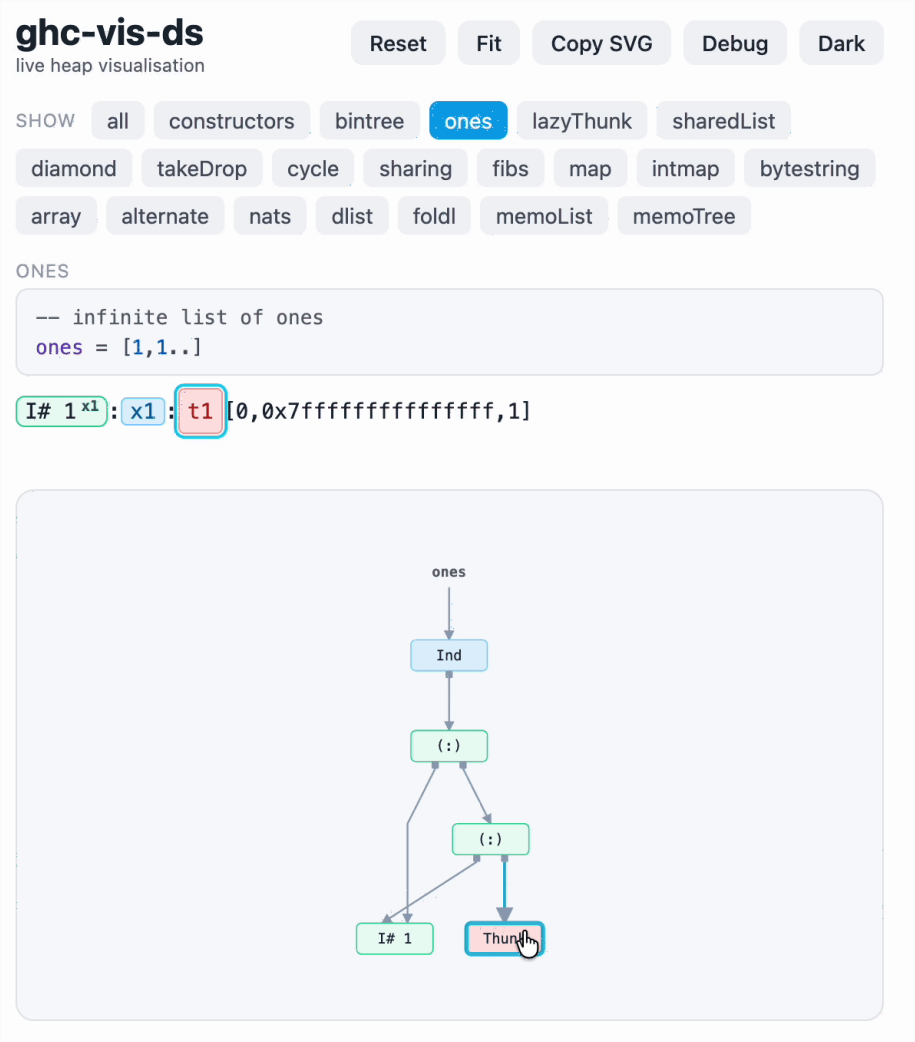
- Обе команды показывают значение, не выполняя отложенные вычисления.
- :sprint заменяет их на _; :print даёт имена, которыми можно воспользоваться.

```
ghci> x = (length [1,3], undefined :: Int)
ghci> :sprint x
x = (_,_)
ghci> :print x
x = ((_t1::Int),(_t2::Int))
ghci> _t1
2
ghci> :sprint x
x = (2,_)
```

- Имена _t1, _t2 генерируются автоматически. Вычисление _t1 обновило общее с x значение.

ghc-vis-ds: лень в браузере

- ghc-vis-ds: значения из GHCi в браузере.
- Щелчок по узлу Thunk выполняет отложенное вычисление до WHNF.
- Видны отложенные вычисления и общие ссылки.
- Онлайн-демо с примерами.



Кадр из README проекта.

Примеры

- Ещё примеры:

Примеры

- Ещё примеры:
- ```
  square x = x*x
  square (square (1+1))
  ```
- ```
  length (take 2 (filter even [1..]))
  ```

Примеры

- Ещё примеры:
- ```
  square x = x*x
  square (square (1+1))
  ```
- ```
  length (take 2 (filter even [1..]))
  ```
- ```
  (&&) :: Bool -> Bool -> Bool
  True  && x = x
  False && x = False
  ```

  ведёт себя как в C автоматически.
  ```

Примеры

- Ещё примеры:
- ```
  square x = x*x
  square (square (1+1))
  ```
- ```
  length (take 2 (filter even [1..]))
  ```
- ```
  (&&) :: Bool -> Bool -> Bool
  True  && x = x
  False && x = False
  ```

  ведёт себя как в C автоматически.
- Как определить вариант, который начинает вычисление с правого операнда?
  ```

Минусы ленивости

- Ленивость позволяет избежать ненужных вычислений.
- Но хранение и обработка отложенных вычислений требуют дополнительных затрат.
- Отложенные вычисления могут занимать больше памяти, чем их результат (а могут и меньше).
- Классический пример:

```
sum [] acc = acc
sum (x:xs) acc = sum xs (x + acc)

sum [1..100] 0
```

Минусы ленивости

- Ленивость позволяет избежать ненужных вычислений.
- Но хранение и обработка отложенных вычислений требуют дополнительных затрат.
- Отложенные вычисления могут занимать больше памяти, чем их результат (а могут и меньше).
- Классический пример:

```
sum [] acc = acc
sum (x:xs) acc = sum xs (x + acc)

sum [1..100] 0 ⇝ sum (1:[2..100]) 0
```

Минусы ленивости

- Ленивость позволяет избежать ненужных вычислений.
- Но хранение и обработка отложенных вычислений требуют дополнительных затрат.
- Отложенные вычисления могут занимать больше памяти, чем их результат (а могут и меньше).
- Классический пример:

```
sum [] acc = acc
sum (x:xs) acc = sum xs (x + acc)

sum [1..100] 0 ⇝ sum (1:[2..100]) 0 ⇝
sum [2..100] ⟨1 + 0⟩
```

Минусы ленивости

- Ленивость позволяет избежать ненужных вычислений.
- Но хранение и обработка отложенных вычислений требуют дополнительных затрат.
- Отложенные вычисления могут занимать больше памяти, чем их результат (а могут и меньше).
- Классический пример:

```
sum [] acc = acc
sum (x:xs) acc = sum xs (x + acc)

sum [1..100] 0 ⇝ sum (1:[2..100]) 0 ⇝
sum [2..100] ⟨1 + 0⟩ ⇝ sum (2:[3..100]) ⟨1 + 0⟩
```

Минусы ленивости

- Ленивость позволяет избежать ненужных вычислений.
- Но хранение и обработка отложенных вычислений требуют дополнительных затрат.
- Отложенные вычисления могут занимать больше памяти, чем их результат (а могут и меньше).
- Классический пример:

```
sum [] acc = acc
sum (x:xs) acc = sum xs (x + acc)

sum [1..100] 0 ⇝ sum (1:[2..100]) 0 ⇝
sum [2..100] ⟨1 + 0⟩ ⇝ sum (2:[3..100]) ⟨1 + 0⟩ ⇝
sum [3..100] ⟨2 + ⟨1 + 0⟩⟩
```

Минусы ленивости

- Ленивость позволяет избежать ненужных вычислений.
- Но хранение и обработка отложенных вычислений требуют дополнительных затрат.
- Отложенные вычисления могут занимать больше памяти, чем их результат (а могут и меньше).
- Классический пример:

```
sum [] acc = acc
sum (x:xs) acc = sum xs (x + acc)

sum [1..100] 0 ⇝ sum (1:[2..100]) 0 ⇝
sum [2..100] ⟨1 + 0⟩ ⇝ sum (2:[3..100]) ⟨1 + 0⟩ ⇝
sum [3..100] ⟨2 + ⟨1 + 0⟩⟩ ⇝ ... ⇝
⟨100 + ... + ⟨1 + 0⟩⟩
```

Минусы ленивости

- Ленивость позволяет избежать ненужных вычислений.
- Но хранение и обработка отложенных вычислений требуют дополнительных затрат.
- Отложенные вычисления могут занимать больше памяти, чем их результат (а могут и меньше).
- Классический пример:

```
sum [] acc = acc
sum (x:xs) acc = sum xs (x + acc)

sum [1..100] 0 ⇝ sum (1:[2..100]) 0 ⇝
sum [2..100] ⟨1 + 0⟩ ⇝ sum (2:[3..100]) ⟨1 + 0⟩ ⇝
sum [3..100] ⟨2 + ⟨1 + 0⟩⟩ ⇝ ... ⇝
⟨100 + ... + ⟨1 + 0⟩⟩ ⇝ 5050
```

Минусы ленивости

- Такой `sum` — по сути `foldl (flip (+))`.
- Из-за этого `foldl` — ловушка, практически всегда нужно либо `foldr`, либо `foldl'` (будет позже).

Минусы ленивости

- Такой `sum` — по сути `foldl (flip (+))`.
- Из-за этого `foldl` — ловушка, практически всегда нужно либо `foldr`, либо `foldl'` (будет позже).
- Что произойдёт в этом примере?

```
length ⟨take 2 ⟨filter (< 2) [1..]⟩⟩
```

Минусы ленивости

- Такой `sum` — по сути `foldl (flip (+))`.
- Из-за этого `foldl` — ловушка, практически всегда нужно либо `foldr`, либо `foldl'` (будет позже).
- Что произойдёт в этом примере?

```
length ⟨take 2 ⟨filter (< 2) [1..]⟩⟩  ⇝
length (1 : ⟨take 1 ⟨filter (< 2) [2..]⟩⟩)
```

Минусы ленивости

- Такой `sum` — по сути `foldl (flip (+))`.
- Из-за этого `foldl` — ловушка, практически всегда нужно либо `foldr`, либо `foldl'` (будет позже).
- Что произойдёт в этом примере?

```
length ⟨take 2 ⟨filter (< 2) [1..]⟩⟩  ⇝
length (1 : ⟨take 1 ⟨filter (< 2) [2..]⟩⟩)  ⇝
1 + ⟨length ⟨take 1 ⟨filter (< 2) [2..]⟩⟩⟩
```

Минусы ленивости

- Такой `sum` — по сути `foldl (flip (+))`.
- Из-за этого `foldl` — ловушка, практически всегда нужно либо `foldr`, либо `foldl'` (будет позже).
- Что произойдёт в этом примере?

```
length ⟨take 2 ⟨filter (< 2) [1..]⟩⟩  ⇝
length (1 : ⟨take 1 ⟨filter (< 2) [2..]⟩⟩)  ⇝
1 + ⟨length ⟨take 1 ⟨filter (< 2) [2..]⟩⟩⟩  ⇝
1 + ⟨length ⟨take 1 ⟨filter (< 2) [3..]⟩⟩⟩
```

Минусы ленивости

- Такой `sum` — по сути `foldl (flip (+))`.
- Из-за этого `foldl` — ловушка, практически всегда нужно либо `foldr`, либо `foldl'` (будет позже).
- Что произойдёт в этом примере?

```
length ⟨take 2 ⟨filter (< 2) [1..]⟩⟩  ⇝
length (1 : ⟨take 1 ⟨filter (< 2) [2..]⟩⟩)  ⇝
1 + ⟨length ⟨take 1 ⟨filter (< 2) [2..]⟩⟩⟩  ⇝
1 + ⟨length ⟨take 1 ⟨filter (< 2) [3..]⟩⟩⟩  ⇝ ...
```

`filter` выдаёт `1`, но второго элемента не будет: все последующие числа не меньше `2`. Поиск продолжается бесконечно; `length` не возвращает даже `1`.

Минусы ленивости

- Такой `sum` — по сути `foldl (flip (+))`.
- Из-за этого `foldl` — ловушка, практически всегда нужно либо `foldr`, либо `foldl'` (будет позже).
- Что произойдёт в этом примере?

- Что же можно сделать?
- Часто компилятор может оптимизировать ленивые вычисления (если знает, что это не изменит результата).
- Или...

`foldr`: ленивый шаг

- Запишем `map (*2)` через `foldr`:

```
step x ys = (x * 2) : ys
```

```
foldr step [] [1..]
```

foldr: ленивый шаг

- Запишем `map (*2)` через `foldr`:

```
step x ys = (x * 2) : ys
```

```
foldr step [] [1..]  ⇝
⟨1 * 2⟩ : ⟨foldr step [] [2..]⟩
```

`foldr`: ленивый шаг

- Запишем `map (*2)` через `foldr`:

```
step x ys = (x * 2) : ys
```

```
foldr step [] [1..]  ⇝
⟨1 * 2⟩ : ⟨foldr step [] [2..]⟩
```

Это уже WHNF: `(:)` не требует вычисления головы и хвоста.

`foldr`: ленивый шаг

- Запишем `map (*2)` через `foldr`:

```
step x ys = (x * 2) : ys
```

Можно потребить только часть результата:

```
sum (take 2 (foldr step [] [1..]))
```

`foldr`: ленивый шаг

- Запишем `map (*2)` через `foldr`:

```
step x ys = (x * 2) : ys
```

Можно потребить только часть результата:

```
sum (take 2 (foldr step [] [1..]))  ⇝
sum (⟨1 * 2⟩ : ⟨take 1 ⟨foldr step [] [2..]⟩⟩)
```

`foldr`: ленивый шаг

- Запишем `map (*2)` через `foldr`:

```
step x ys = (x * 2) : ys
```

Можно потребить только часть результата:

```
sum (take 2 (foldr step [] [1..]))  ⇝
sum (⟨1 * 2⟩ : ⟨take 1 ⟨foldr step [] [2..]⟩⟩)  ⇝
sum [⟨1 * 2⟩, ⟨2 * 2⟩]
```

- При раскрытии `take` произведения остаются отложенными; `sum` требует их значения.

`foldr`: ленивый шаг

- Запишем `map (*2)` через `foldr`:

```
step x ys = (x * 2) : ys
```

Можно потребить только часть результата:

```
sum (take 2 (foldr step [] [1..]))  ⇝
sum (⟨1 * 2⟩ : ⟨take 1 ⟨foldr step [] [2..]⟩⟩)  ⇝
sum [⟨1 * 2⟩, ⟨2 * 2⟩]  ⇝
sum [2, 4]
```

- При раскрытии `take` произведения остаются отложенными; `sum` требует их значения.

`foldr`: ленивый шаг

- Запишем `map (*2)` через `foldr`:
  ```
  step x ys = (x * 2) : ys
  ```

Можно потребить только часть результата:

```
sum (take 2 (foldr step [] [1..]))  ⇝
sum (⟨1 * 2⟩ : ⟨take 1 ⟨foldr step [] [2..]⟩⟩)  ⇝
sum [⟨1 * 2⟩, ⟨2 * 2⟩]  ⇝
sum [2, 4]  ⇝ 6
```

- При раскрытии `take` произведения остаются отложенными; `sum` требует их значения.
- Вызов `foldr step [] [3..]` не потребуется.

`foldr`: ленивый шаг

- Запишем `map (*2)` через `foldr`:
  ```
  step x ys = (x * 2) : ys
  ```

Можно потребить только часть результата:

```
sum (take 2 (foldr step [] [1..]))  ⇝
sum (⟨1 * 2⟩ : ⟨take 1 ⟨foldr step [] [2..]⟩⟩)  ⇝
sum [⟨1 * 2⟩, ⟨2 * 2⟩]  ⇝
sum [2, 4]  ⇝ 6
```

- При раскрытии `take` произведения остаются отложенными; `sum` требует их значения.
- Вызов `foldr step [] [3..]` не потребуется.
- А `foldr (*) 0 [1..]` не возвращает результат: `(*)` требует вычислить рекурсивный результат.

Строгие вычисления в Haskell: `seq`

- Мы можем управлять ленивостью явно.
- Базовый инструмент для этого: «волшебная» (её нельзя реализовать на самом Haskell) функция `seq :: a -> b -> b`.

Строгие вычисления в Haskell: `seq`

- Мы можем управлять ленивостью явно.
- Базовый инструмент для этого: «волшебная» (её нельзя реализовать на самом Haskell) функция `seq :: a -> b -> b`.
- `seq` e_1 e_2 возвращает e_2, но требует вычисления e_1 до WHNF. Порядок вычисления аргументов не гарантируется.

Строгие вычисления в Haskell: `seq`

- Мы можем управлять ленивостью явно.
- Базовый инструмент для этого: «волшебная» (её нельзя реализовать на самом Haskell) функция `seq :: a -> b -> b`.
- `seq` e_1 e_2 возвращает e_2, но требует вычисления e_1 до WHNF. Порядок вычисления аргументов не гарантируется.
- Обычно e_1 — переменная в e_2.
- Пример использования:
  ```
  f $! x = seq x (f x)
  ```
- Это аналог `$`, только вычисляющий аргумент до WHNF перед вызовом функции.

`sum` через `seq`

- Теперь можем определить `sum` так:

`sum` через `seq`

- Теперь можем определить `sum` так:

```
sum [] acc = acc
sum (x:xs) acc = sum xs $! x + acc
```

- Или так:

sum через seq

- Теперь можем определить sum так:

```
sum [] acc = acc
sum (x:xs) acc = sum xs $! x + acc
```

- Или так:

```
sum [] acc = acc
sum (x:xs) acc = let a1 = x + acc in
                 a1 `seq` sum xs a1
```

- И процесс вычисления:

```
sum [1..100] 0 ⇝ sum (1:[2..100]) 0
```

sum через seq

- Теперь можем определить sum так:

```
sum [] acc = acc
sum (x:xs) acc = sum xs $! x + acc
```

- Или так:

```
sum [] acc = acc
sum (x:xs) acc = let a1 = x + acc in
                 a1 `seq` sum xs a1
```

- И процесс вычисления:

```
sum [1..100] 0 ⇝ sum (1:[2..100]) 0  ⇝
let a1 = ⟨1 + 0⟩ in a1 `seq` sum [2..100] a1
```

sum через seq

- Теперь можем определить sum так:

```
sum [] acc = acc
sum (x:xs) acc = sum xs $! x + acc
```

- Или так:

```
sum [] acc = acc
sum (x:xs) acc = let a1 = x + acc in
                 a1 `seq` sum xs a1
```

- И процесс вычисления:

```
sum [1..100] 0 ⇝ sum (1:[2..100]) 0  ⇝
let a1 = ⟨1 + 0⟩ in a1 `seq` sum [2..100] a1  ⇝
sum [2..100] 1 ⇝ sum (2:[3..100]) 1
```

sum через seq

- Теперь можем определить sum так:

```
sum [] acc = acc
sum (x:xs) acc = sum xs $! x + acc
```

- Или так:

```
sum [] acc = acc
sum (x:xs) acc = let a1 = x + acc in
                 a1 `seq` sum xs a1
```

- И процесс вычисления:

```
sum [1..100] 0 ⇝ sum (1:[2..100]) 0  ⇝
let a1 = ⟨1 + 0⟩ in a1 `seq` sum [2..100] a1  ⇝
sum [2..100] 1 ⇝ sum (2:[3..100]) 1  ⇝
let a1 = ⟨2 + 1⟩ in a1 `seq` sum [3..100] a1
```

sum через seq

- Теперь можем определить sum так:

```
sum [] acc = acc
sum (x:xs) acc = sum xs $! x + acc
```

- Или так:

```
sum [] acc = acc
sum (x:xs) acc = let a1 = x + acc in
                 a1 `seq` sum xs a1
```

- И процесс вычисления:

```
sum [1..100] 0 ⇝ sum (1:[2..100]) 0  ⇝
let a1 = ⟨1 + 0⟩ in a1 `seq` sum [2..100] a1  ⇝
sum [2..100] 1 ⇝ sum (2:[3..100]) 1  ⇝
let a1 = ⟨2 + 1⟩ in a1 `seq` sum [3..100] a1  ⇝
⇝ ... ⇝ 5050
```

- Заметьте, что список [1..100] никогда не появляется целиком в памяти!

foldl'

- Мы знаем, что сумма списка — частный случай свёртки.
- Для функций `product`, `minimum`, `maximum` имеем ровно те же проблемы!
- Можем определить вариант `foldl` (с GHC 9.10 он уже есть в `Prelude`):

```
foldl' :: (a -> b -> a) -> a -> [b] -> a
foldl' f acc []     = acc
foldl' f acc (x:xs) = let acc' = f acc x
                      in acc' `seq` foldl' f acc' xs
```

foldl'

- Мы знаем, что сумма списка — частный случай свёртки.
- Для функций `product`, `minimum`, `maximum` имеем ровно те же проблемы!
- Можем определить вариант `foldl` (с GHC 9.10 он уже есть в `Prelude`):

```haskell
foldl' :: (a -> b -> a) -> a -> [b] -> a
foldl' f acc []     = acc
foldl' f acc (x:xs) = let acc' = f acc x
                      in acc' `seq` foldl' f acc' xs
```

- ```haskell
  sum xs = foldl' (+) 0 xs
  product xs = foldl' (*) 1 xs
  minimum xs = foldl1' min xs  -- foldl1’ из Data.List
  ...
  ```
  ```

Строгие переменные в образцах

- В `foldl'` аккумулятор вычисляется только до WHNF. Если мы вычисляем среднее так

```
mean :: [Double] -> Double
mean xs = s' / fromIntegral l'
  where (s', l') = foldl' step (0, 0) xs
        step (s, l) x = (s + x, l + 1)
```

  то в `s` и `l` снова накапливаются вычисления!

Строгие переменные в образцах

- В `foldl'` аккумулятор вычисляется только до WHNF. Если мы вычисляем среднее так

```
mean :: [Double] -> Double
mean xs = s' / fromIntegral l'
  where (s', l') = foldl' step (0, 0) xs
        step (s, l) x = (s + x, l + 1)
```

  то в `s` и `l` снова накапливаются вычисления!

```
foldl' step (0, 0) [1, 2]
```

Строгие переменные в образцах

- В `foldl'` аккумулятор вычисляется только до WHNF. Если мы вычисляем среднее так

```
mean :: [Double] -> Double
mean xs = s' / fromIntegral l'
  where (s', l') = foldl' step (0, 0) xs
        step (s, l) x = (s + x, l + 1)
```

  то в `s` и `l` снова накапливаются вычисления!

```
foldl' step (0, 0) [1, 2] ⇝
foldl' step (⟨0 + 1⟩, ⟨0 + 1⟩) [2]
```

Строгие переменные в образцах

- В `foldl'` аккумулятор вычисляется только до WHNF. Если мы вычисляем среднее так

```
mean :: [Double] -> Double
mean xs = s' / fromIntegral l'
  where (s', l') = foldl' step (0, 0) xs
        step (s, l) x = (s + x, l + 1)
```

  то в `s` и `l` снова накапливаются вычисления!

```
foldl' step (0, 0) [1, 2]  ⇝
foldl' step (⟨0 + 1⟩, ⟨0 + 1⟩) [2]  ⇝
foldl' step (⟨⟨0 + 1⟩ + 2⟩, ⟨⟨0 + 1⟩ + 1⟩) []
```

- Здесь аккумулятор — пара, и её вычисление до WHNF не вычисляет компонент.

Строгие переменные в образцах (продолжение)

- Сделаем `s` и `l` строгими:
  ```
  step (!s, !l) x = (s + x, l + 1)
  ```
- Это превращается в
  ```
  step (s, l) x = s `seq` l `seq` (s + x, l + 1)
  ```
  т.е. ! вычисляет значение (до WHNF) при сопоставлении. Этого хватает: `foldl'` приводит результат `step` к WHNF, а для этого `step` сопоставляет предыдущую пару с образцом и вычисляет `s` и `l`.
- Полные правила перевода довольно сложные.

Строгие поля в типах данных

- Другой способ решить ту же проблему:

```
data SPair a b = SPair !a !b

mean :: [Double] -> Double
mean xs = s' / fromIntegral l'
  where SPair s' l' = foldl' step (SPair 0 0) xs
        step (SPair s l) x = SPair (s + x) (l + 1)
```

Строгие поля в типах данных

- Другой способ решить ту же проблему:

```
data SPair a b = SPair !a !b

mean :: [Double] -> Double
mean xs = s' / fromIntegral l'
  where SPair s' l' = foldl' step (SPair 0 0) xs
        step (SPair s l) x = SPair (s + x) (l + 1)
```

- Восклицательный знак означает, что любое вычисление значения с этим конструктором вычислит и эти поля (до WHNF).
- То есть конструктор работает как

```
SPair a b = a `seq` b `seq` SPair' a b
```

где `SPair'` — конструктор, который получится без строгих полей.

Дополнительное чтение

- Parameter-Passing Mechanisms (OpenDSA): способы передачи параметров, включая макроподстановку, передачу по имени и по необходимости.
- Why Functional Programming Matters (обзор статьи, написанной до появления собственно Haskell)
- All About Strictness
- The Incomplete Guide to Lazy Evaluation
- Laziness в Haskell Wikibook

Дополнительное чтение: память

- Joachim Breitner, Thunks, Sharing, Laziness: The Haskell Heap Visualized: видеодемонстрация отложенных вычислений, общих ссылок и циклических структур с помощью ghc-vis.
- The Programs of Consistent Lethargy (Haskell Optimization Handbook): ленивость и расход памяти; почему `foldl'` не вычисляет поля пары до конца.